

DesignCorrection-R1: Learning to Reason for Graphic Design Correction

SHUNSUKE KITADA^{1,a)} SHOMA IWAI^{1,b)} RYOTA YOSHIHASHI^{1,c)}
ATSUKI OSANAI^{1,d)}

Abstract

Creative graphic design involves iterative refinement over structured elements, where subtle spatial and typographic defects can harm readability and visual quality. Although recent vision-language models can generate plausible layouts, reliable fine-grained correction remains difficult due to constraint violations and the ambiguity of valid edits. We introduce DesignCorrection-R1, a reasoning-oriented vision-language policy that performs local, intent-preserving repair by generating minimal executable edit programs over a structured element-level action space. The model is trained with a staged curriculum for layout and style repair and optimized using DeepSeek-R1-inspired reinforcement learning with constraint-based and minimal-edit rewards. Preliminary results on synthetic layout defects show improved issue localization, supporting the effectiveness of structured reasoning for design correction.

1. Introduction

Creative graphic design generation [7], [21] aims to synthesize complete visual artifacts by jointly deciding the arrangement and appearance of multiple elements, layout, typography, colors, and layering, under implicit design principles. This umbrella covers a broad range of applications, including poster and template design [1], [5], [10], magazine pages [20], advertisement/banner creatives [19], and prompt-driven text-to-design draft synthesis [15]. Compared to generic image generation [3], [11], [12], these tasks require structured reasoning over discrete objects (e.g., text blocks, logos, underlays) and their fine-grained attributes, and are therefore naturally aligned with *editable* representations rather than a single raster image.

Despite this, many modern pipelines remain *raster-centric* [2], [16]: they can generate visually plausible drafts, but the output is a bitmap and does not directly support element-level revisions. In contrast, real creative workflows are iterative and operate on structured design states (layer stacks / element graphs) where each object exposes geome-

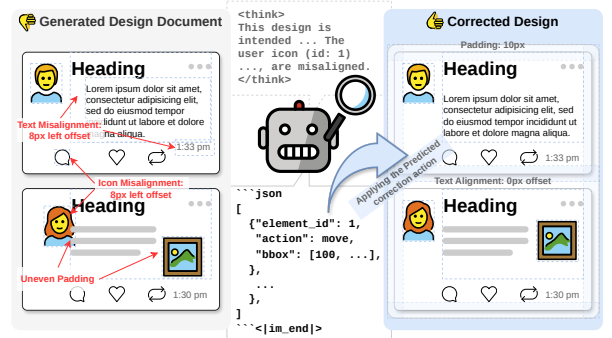


Fig. 1 Overview of DesignCorrection-R1. From a rendered design and its structured element state, the policy generates a compact edit program in an API-aligned action space. RL with constraint-based and minimal-edit rewards encourages precise, intent-preserving local repair.

try and style parameters. This mismatch motivates *vector-centric* or structured generation and editing, and has inspired work that explicitly treats re-editability as a first-class requirement [14], [18]. However, editability alone does not solve the last-mile refinement problem: designers still spend substantial effort fixing local defects that harm readability and perceived quality.

A growing body of evidence suggests that **strong generation does not imply strong edit/repair**. Benchmarks that explicitly evaluate editing and repair (e.g., software issue resolution and UI/front-end repair) show that models capable of producing plausible drafts often struggle to identify *what* to change and to apply minimal modifications under constraints [8], [17]. This gap is particularly pronounced in design, where actionable defects are frequently *perceptually subtle but quality-critical*, a few-pixel misalignment within a text, underlay group, slightly uneven padding that breaks visual rhythm, or a small contrast/opacity drift that makes text hard to read. Humans notice such deviations at a glance, yet current multimodal models often struggle with fine-grained spatial relations and precise grounding [9].

Design correction is also intrinsically *underdetermined*: for a flawed draft there is rarely a single correct edit. Multiple local fixes can satisfy constraints while preserving intent, and global redesign further expands the space of acceptable outcomes, making automatic rewards unstable. We therefore focus on **local, intent-preserving repair** over

¹ LY Corporation

a) s.kitada@lycorp.co.jp

b) shoiwai@lycorp.co.jp

c) ryoshiha@lycorp.co.jp

d) atsuki.osanai@lycorp.co.jp

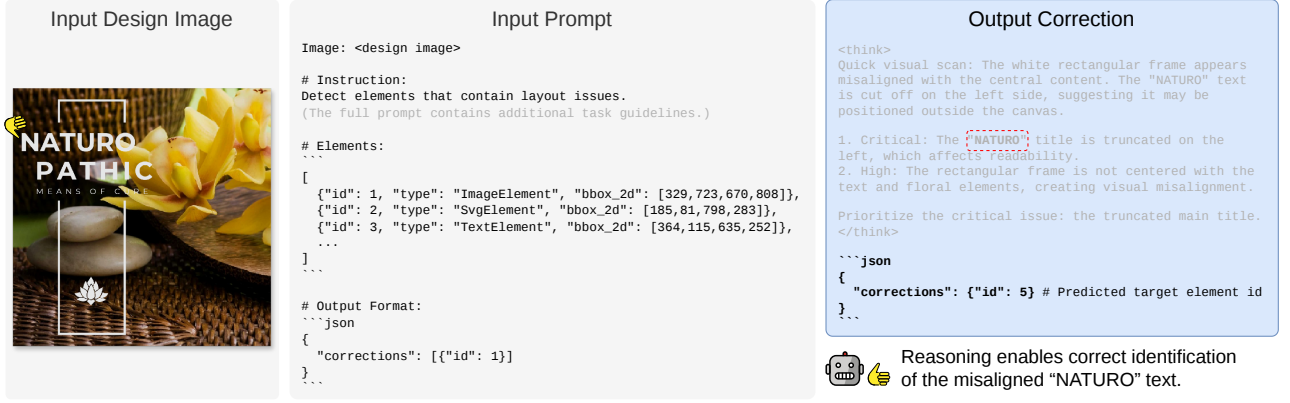


Fig. 2 Example of the input and output of DesignCorrection-R1. A design image is provided together with a structured prompt containing design elements and their bounding boxes. The model analyzes the layout, identifies elements with potential design problems through step-by-step reasoning, and outputs the target element ID requiring correction in a structured JSON format.

structured design states: correcting small groups of related elements (e.g., text blocks with underlays) while keeping the global composition stable.

In this paper, we propose **DesignCorrection-R1**, a reasoning-oriented vision-language policy for local design repair, as shown in Fig. 1. Given a design image and its structured element specification, the model generates a short *executable edit program* defined in an editing API space. Our central idea is to cast correction as a deliberative process that mirrors expert workflow: diagnose which elements violate constraints, decide the appropriate edit type, and parameterize minimal, intent-preserving adjustments. This structured decomposition is particularly suited to thinking-capable VLMs. We train the policy with a two-track curriculum that separately develops *layout repair* and *style repair*, followed by a unified stage. To handle the intrinsic non-uniqueness of valid fixes, we adopt reinforcement learning inspired by DeepSeek-R1 [4], [13] style reasoning optimization, using programmatic rewards that combine constraint satisfaction, minimal-edit regularization, and optional multi-reference supervision to incentivize explicit, stepwise correction behavior rather than surface-level generation.

2. Method

DesignCorrection-R1 is a vision-language policy for local, intent-preserving repair of structured graphic designs (Fig. 2). Given a rendered image I and an editable element set E_0 , the model predicts a compact JSON edit program A that modifies geometry or style at the element level. In contrast to layout generation or global re-editing, our objective is minimal correction over an existing design state. Because valid fixes are inherently non-unique, we train the policy with reinforcement learning using programmatic rewards based on constraint improvement and edit sparsity.

2.1 Problem Formulation

A design is represented as $E = \{e_i\}_{i=1}^N$, where each element stores a type label, z-index, bounding box b_i , and optional style attributes. The policy outputs an edit set

$A = \{a_j\}_{j=1}^M$, and the corrected state is

$$E_1 = \text{Apply}(E_0, A). \quad (1)$$

We focus on local geometric and stylistic updates without adding, removing, or rewriting elements, which keeps evaluation and reward computation tractable.

2.2 Synthetic Corruption for Training

As real correction traces are scarce and multi-modal, we construct training pairs by corrupting clean designs. Starting from E_{clean} , we sample $K \in \{1, 2, 3\}$ corruption operators to obtain a noisy state E_0 . Layout corruptions include alignment breaks, padding violations, overlap creation, z-order text hiding, and out-of-bounds shifts. Style corruptions include font size, spacing, alignment, opacity, angle, and out-of-bounds shifts. Style corruptions include contrast drift. Geometric perturbations are grid-snapped in normalized space to make inversion learnable. We filter samples by a constraint-based difficulty score and store lightweight metadata for curriculum rewards and tool shaping.

Constraint-based Scoring. We define a scoring function $S(E) \in [0, 1]$ that measures layout quality. The score penalizes overflow beyond the canvas, element overlap, insufficient padding, and misalignment across detected alignment groups. Training rewards depend on score improvement $S(E_1) - S(E_0)$ rather than exact matching, which naturally accommodates non-unique valid corrections.

Minimal-Edit Multi-reference Regularization. To discourage unnecessary edits, we penalize edit magnitude:

$$C(E_0, E_1) = \frac{1}{N} \sum_{i=1}^N \left(\frac{|\Delta c_{x,i}| + |\Delta c_{y,i}|}{W} + \frac{|\Delta w_i| + |\Delta h_i|}{2W} \right). \quad (2)$$

We optionally incorporate a multi-reference bonus that rewards proximity to any valid reference layout. Given reference designs $\{E_{\text{ref}}^{(k)}\}$, we define

$$B_{\text{ref}}(E_1) = \exp(-\min_k d(E_1, E_{\text{ref}}^{(k)})/\tau) \quad (3)$$

Table 1 Correction results for Layout Stage 1 / Phase 1. Best and second-best results are in **bold** and underline. For models without structured output, invalid JSON predictions are treated as zero-score outputs when computing precision, recall, and F1. [†]OpenAI API structured output is used. [‡]No structured output function is used.

Method	Model	Valid JSON $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$
Random Corrector		N/A	0.1300	0.1183	0.1239
Heuristic Corrector		N/A	0.1745	0.1580	0.1659
Zero-shot VLM	gpt-5-mini-2025-08-07	[†] 1.000	0.1616	0.4221	0.2337
	gpt-5-2025-08-07	[†] 1.000	<u>0.1948</u>	<u>0.5414</u>	<u>0.2865</u>
	Qwen/Qwen3-VL-32B-Thinking	0.317	0.0537	0.0694	0.0605
DesignCorrection-R1 (Ours)	Qwen/Qwen3-VL-32B-Thinking	[‡] 1.000	0.2421	0.5532	0.3438

where d is the mean L1 distance between normalized bounding boxes. The clean design is always included as a reference.

We optimize the policy using Group Relative Policy Optimization (GRPO) [13]. For each prompt, multiple completions are sampled and ranked by a programmatic:

$$r = r_{\text{fmt}} + \alpha(S(E_1) - S(E_0)) - \beta C(E_0, E_1) + \gamma B_{\text{ref}}, \quad (4)$$

where r_{fmt} enforces valid JSON output. To stabilize learning, we apply a three-phase output curriculum: phase1 predicts target element ids, phase2 predicts element ids and action types, and phase3 predicts full parameterized edits. Early phases include auxiliary rewards aligned with corruption logs, while the final phase emphasizes constraint improvement and minimal edits.

Inference. The model receives I and the serialized element list E_0 . It outputs a JSON edit program, which is parsed and applied to obtain the corrected design state E_1 . Optional per-element crops can be appended to the visual input to improve sensitivity to subtle local defects.

3. Experiments

We report a preliminary evaluation focusing on Layout Stage1, Phase1, the first stage of our curriculum where the model performs *issue localization* and outputs only the ids of elements requiring correction. This setting isolates the core bottleneck of last-mile repair, *detecting which elements are problematic*, before evaluating action selection or parameterized edits.

3.1 Data: Synthetic Layout Defects

Our data is constructed from the Crello v5.0.0 [18]^{*1} design dataset, which provides template-based graphic designs with structured element annotations. We build training and validation data by corrupting clean designs with a single layout operator ($K=1$) and enforcing in-bounds geometry. Following our Stage1 preset, we use four layout corruptions: **alignment_break**, **padding_break**, **overlap_create**, and **z_order_hide_text**. We use the **easy** difficulty preset, where the maximum perturbation corresponds to 4–8 grid steps under our normalized $[0, 1000]$ coordinate system (grid width 8). Style noise is disabled in this stage to avoid confounding geometry-based localization. Designs are rendered with text enabled and resized to a maximum of 1024 pixels on the longer side.

^{*1} <https://huggingface.co/datasets/cyberagent/crello>

3.2 Models and Training

We start from a thinking-capable VLM and optimize it with GRPO using parameter-efficient fine-tuning (LoRA) [6]. Note that the vision encoder is updated with full fine-tuning rather than being frozen, as the task requires adapting visual representations to subtle layout inconsistencies in graphic designs. The language model is adapted using LoRA. In Phase1, the output is a JSON array of objects containing only **element_id**. We include a format reward to enforce valid JSON and an element localization reward. For rollouts, we sample multiple completions per prompt and update the policy with within-group relative advantages.

3.3 Baselines and Metrics

Baselines. We compare our method with four types of baselines. **Random** randomly selects element IDs from the element list, providing a lower bound for the task. **Heuristic** applies simple rule-based checks based on element geometry and layout statistics (e.g., spacing inconsistencies or overlaps). **Zero-shot VLM** evaluates vision-language models without task-specific training. Given the rendered design and the element list, the models are prompted to output problematic **element_ids** in JSON format. We test **Qwen3-VL-32B-Thinking**, **gpt-5-mini**, and **gpt-5**. **DesignCorrection-R1** uses the same prompting interface and operates under the same zero-shot inference setting. The difference is that the base model (**Qwen3-VL-32B-Thinking**) is further trained using the proposed staged curriculum.

JSON validity. Since the model must produce structured outputs, we report *Valid JSON*, the fraction of responses that can be parsed into the expected Phase1 format. In this phase, the output is a top-level JSON array whose items contain only **element_id**. Outputs that cannot be parsed into this structure are counted as invalid.

Element Precision/Recall/F1. In Layout Stage1 / Phase1, the model predicts a set of problematic **element_ids**. Evaluation compares predicted and ground-truth element sets. Precision, Recall, and F1 are computed at the element level and aggregated over the evaluation set.

4. Results and Discussion

4.1 Quantitative Evaluation

Table 1 reports validation performance for Layout Stage1

Input image	Good: The misaligned elements are predicted correctly.	Fair: The misaligned elements are predicted, but additional unnecessary elements are also predicted.
	<pre> <think> Quick scan: The "HOME" text (id 7) is oversized and overlaps the "MONEY" text (id 4), making it hard to read. This is a high-priority issue due to significant overlap hiding key text. The "from" text (id 6) is also partially occluded by the "MONEY" text, but the main problem is the "HOME" overlap. The design appears broken, not intentional. </think> ```json [{"element_id": 7}] ``` < im_end > </pre>	<pre> <think> Quick scan: The text elements (3,4,6,7) overlap heavily and are misaligned. The "MONEY" text (4) is partially hidden by "HOME" (7), which is larger and z-indexed lower. This causes critical occlusion of key text. 1. "HOME" (7) is behind "MONEY" (4) but is larger and covers it-critical issue. 2. "MONEY" (4) is misaligned with "MAKE" (3) and "from" (6), breaking the design structure. 3. "from" (6) is too small and mispositioned under "MONEY", causing visual inconsistency. Critical: "HOME" (7) occludes "MONEY" (4) due to z-order and size. High: "MONEY" and "from" misalignment. Medium: "from" spacing issue. </think> ```json [{"element_id": 4}, {"element_id": 7}] ``` < im_end > </pre>

Fig. 3 Examples of reasoning for element-level issue detection. The model analyzes element relationships and predicts problematic elements. The good example identifies the issue, while the fair example includes additional unnecessary predictions.

Table 2 Ablation on vision encoder adaptation.

Variant	Valid JSON $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$
 Vision frozen	1.000	0.152	0.298	0.201
 Vision tuned	1.000	0.242	0.553	0.344

/ Phase1, which evaluates the ability to *localize problematic elements* given a rendered design and its element list.

The heuristic corrector improves over the random baseline (F1: 0.1659 vs. 0.1239), showing that simple rule-based signals capture some layout inconsistencies, but remain limited without global visual reasoning.

Zero-shot VLMs show a different behavior. **gpt-5-mini** and **gpt-5** achieve higher recall (0.4221 and 0.5414) than heuristic methods but with lower precision, indicating frequent over-prediction. In contrast, the open-source **Qwen3-VL-32B-Thinking** performs poorly in the zero-shot setting, with low JSON validity (0.317) and F1 (0.0605), highlighting the difficulty of structured design reasoning without task-specific training.

Our DesignCorrection-R1 substantially improves performance when trained with the proposed curriculum. Using the same base model (**Qwen3-VL-32B-Thinking**), it achieves the best F1 score (**0.3438**) with higher precision (0.2421) and strong recall (0.5532). The model also consistently produces valid structured outputs (Valid JSON: 1.000), which is important for executable design edits.

Table 2 compares full fine-tuning of the vision encoder with a frozen-vision variant, while keeping LoRA adaptation on the language model in both cases. Freezing the vision encoder leads to a large drop in Recall and F1. This result suggests that layout defect localization depends on adapting visual features to subtle geometric inconsistencies in rendered designs, and that language-side adaptation alone is insufficient for this task.

4.2 Qualitative Evaluation

Figure 3 illustrates representative reasoning traces pro-

duced by the model during element-level issue detection. Given a rendered design image, the model examines relationships among design elements, including overlap, alignment, and relative scale, and infers which elements disrupt the layout structure. It then outputs the detected problematic elements as a set of `element_ids`.

The good example correctly localizes the element responsible for the layout issue. In contrast, the fair example includes the correct element but also predicts additional elements. This behavior reflects the model’s reasoning process over element interactions when identifying layout problems.

5. Conclusion

We presented DesignCorrection-R1, a vision-language policy for structured graphic design correction that operates on editable design representations rather than raster images. The model predicts minimal element-level edits given a rendered design and its element list. In this paper, we focused on Layout Stage1 / Phase1 and evaluated the model’s ability to localize problematic elements. The results show that the proposed training framework improves element localization compared with heuristic methods and zero-shot VLM baselines while producing valid structured outputs.

DesignCorrection-R1 is designed as part of a broader staged training pipeline that extends beyond this initial localization task. Later phases introduce action prediction and parameterized edits, and the style stage expands the correction space to typography and visual attributes. While experiments for these components have been conducted, we report only the first stage here due to space constraints. Ultimately, we aim to integrate these capabilities with tool use, enabling models to analyze designs, execute structured edits, and iteratively refine results. We believe this direction will lead toward agentic design correction systems that support practical, interactive design workflows.

References

- [1] Chen, H., Xu, X., Li, W., Ren, J., Ye, T., Liu, S., Chen, Y.-C., Zhu, L. and Wang, X.: Posta: A go-to framework for customized artistic poster generation, *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 28694–28704 (2025).
- [2] Chen, J., Jiang, H., Wang, Y., Wu, K., Li, J., Zhang, C., Yanai, K., Chen, D. and Yuan, Y.: Prismlayers: Open data for high-quality multi-layer transparent image generative models, *arXiv preprint arXiv:2505.22523* (2025).
- [3] Fortin, A., Vernade, G., Kampf, K. and Reshi, A.: Introducing Gemini 2.5 Flash Image (aka nano-banana), Google Developers Blog (2025).
- [4] Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X. et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, *arXiv preprint arXiv:2501.12948* (2025).
- [5] Hsu, H. Y., He, X., Peng, Y., Kong, H. and Zhang, Q.: Posterlayout: A new benchmark and approach for content-aware visual-textual presentation layout, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6018–6026 (2023).
- [6] Hu, E. J., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W. et al.: LoRA: Low-Rank Adaptation of Large Language Models, *International Conference on Learning Representations*.
- [7] Huang, D., Guo, J., Sun, S., Tian, H., Lin, J., Hu, Z., Lin, C.-Y., Lou, J.-G. and Zhang, D.: A survey for graphic design intelligence, *arXiv preprint arXiv:2309.01371* (2023).
- [8] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. and Narasimhan, K. R.: SWE-bench: Can Language Models Resolve Real-world Github Issues?, *The Twelfth International Conference on Learning Representations* (2024).
- [9] Liu, J., Liu, Z., Cen, Z., Zhou, Y., Zou, Y., Zhang, W., Jiang, H. and Ruan, T.: Can Multimodal Large Language Models Understand Spatial Relations?, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 620–632 (2025).
- [10] Liu, J., Zhang, P., Zhang, Y., Yan, P., Zhou, H., Zhou, X., Guo, F. and Jin, L.: PosterVerse: A Full-Workflow Framework for Commercial-Grade Poster Generation with HTML-Based Scalable Typography, *arXiv preprint arXiv:2601.03993* (2026).
- [11] OpenAI: GPT Image 1.5 Model Documentation, OpenAI API Documentation (2025).
- [12] Rombach, R., Blattmann, A., Lorenz, D., Esser, P. and Ommer, B.: High-resolution image synthesis with latent diffusion models, *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10684–10695 (2022).
- [13] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. et al.: Deepseekmath: Pushing the limits of mathematical reasoning in open language models, *arXiv preprint arXiv:2402.03300* (2024).
- [14] Suzuki, T., Liu, K.-J., Inoue, N. and Yamaguchi, K.: Layerd: Decomposing raster graphic designs into layers, *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 17783–17792 (2025).
- [15] Wang, Z., Bao, J., Gu, S., Chen, D., Zhou, W. and Li, H.: Designdiffusion: High-quality text-to-design image generation with diffusion models, *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 20906–20915 (2025).
- [16] Wu, C., Li, J., Zhou, J., Lin, J., Gao, K., Yan, K., Yin, S.-m., Bai, S., Xu, X., Chen, Y. et al.: Qwen-image technical report, *arXiv preprint arXiv:2508.02324* (2025).
- [17] Xiao, J., Wang, M., Lam, M. H., Wan, Y., Liu, J., Huo, Y. and Lyu, M. R.: Designbench: A comprehensive benchmark for mllm-based front-end code generation, *arXiv preprint arXiv:2506.06251* (2025).
- [18] Yamaguchi, K.: Canvasvae: Learning to generate vector graphic documents, *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5481–5489 (2021).
- [19] Yoshitake, K., Hosono, K., Kobayashi, K. and Nakata, K.: Content-Aware Ad Banner Layout Generation with Two-Stage Chain-of-Thought in Vision Language Models, *arXiv preprint arXiv:2512.12596* (2025).
- [20] Zheng, X., Qiao, X., Cao, Y. and Lau, R. W.: Content-aware generative modeling of graphic design layouts, *ACM Transactions on Graphics (TOG)*, Vol. 38, No. 4, pp. 1–15 (2019).
- [21] Zou, X., Zhang, W. and Zhao, N.: From Fragment to One Piece: A Review on AI-Driven Graphic Design, *Journal of Imaging*, Vol. 11, No. 9, p. 289 (2025).